

Copy of [Original Document](#) so I can make comments

Change Log

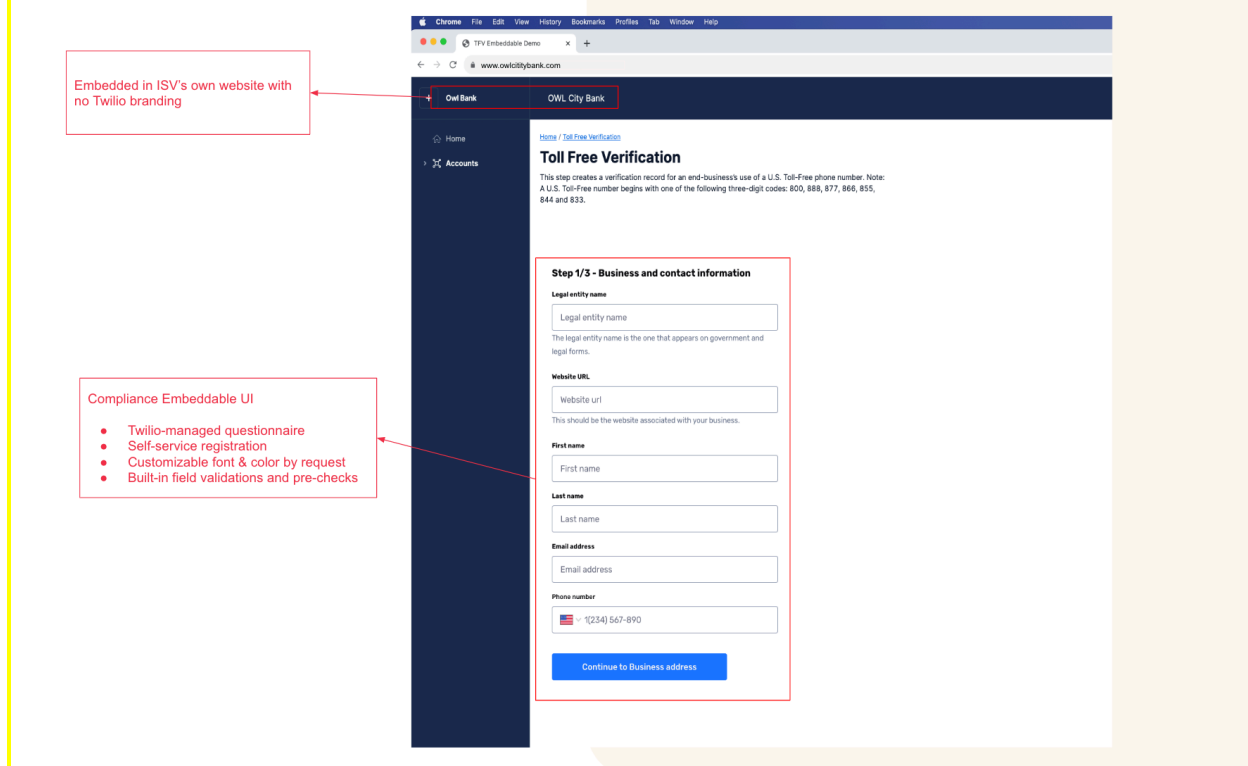
Version	Date	Changes
1.0	Mar 17, 2026	API specs created

Overview

The Compliance Embeddable is a white-label embed offering designed to seamlessly integrate into an Independent Software Vendor's (ISV) existing portal or web application. The user interface is tailored specifically for the ISV tenant use case with a form containing all the necessary questions required by a verification or registration, allowing ISVs' end customers to submit their compliance information via self-service. Twilio manages the questions that are asked in the UI, but there is no Twilio branding on the Compliance Embeddable. The UI can be customized by the Twilio team to align with your design standards, including color palette and typeface.

After reading this document, you will be able to integrate the Compliance Embeddable into your web application to allow self-service alphanumeric sender ID registrations for your end customers in Australia.

Compliance Embeddable UI



The Compliance Embeddable requires two integrations:

1. **Server-side** integration to invoke the Twilio Initialize ComplianceInquiry APIs to initialize the ComplianceInquiry and get back a session token and inquiry ID.
2. **Client-side** integration to embed the ComplianceInquiry UI in your existing web application using an iframe, utilizing the returned session token and inquiry ID.

This onboarding guide provides a step-by-step walkthrough on how to integrate the Compliance Embeddable into an ISV's web application, allowing self-service alphanumeric sender ID registrations for your end customers in Australia.

If you have questions about this product, check out the [Compliance Embeddable FAQ](#) or contact Twilio to the pilot feedback form.

Prerequisite

Before you can use the compliance embeddable for alphanumeric sender IDs in Australia, all Independent Software Vendors (ISVs) must have opted-in to partnering with Twilio to register the sender IDs. You can do so by sending an email to senderid@twilio.com.

For ISVs with ABN, the ACMA requires you to independently [register as a "participating telco" on ACMA Assist](#) before Twilio can submit registrations on ACMA Assist on your behalf.

Understanding the Sender ID Registration Flow in Australia

Starting July 1, 2026, all alphanumeric sender IDs used for SMS in Australia need to be registered on the ACMA Sender ID Register (ACMA Assist).

Important: Any sender IDs that aren't registered by July 1, 2026 will be modified to display as "Unverified" to message recipients.

Before starting, make sure you have the following information ready. For Independent Software Vendor (ISV) registering sender IDs on behalf of your customers, the following information is of your tenants / customers.

- **Business Proof:** An up-to-date commercial registration extract (e.g., ASIC company extract) showing the business name, address, registration number, and officers. Accepted formats: JPG, PNG, or PDF (max 5 MB).
- **Sender ID Proof:** Evidence connecting the brand to the requested sender ID. For non-Australia based businesses, you may be required to provide a document such as a trademark certificate or domain registration if the sender ID does not match the business name on the commercial registration extract.
- **ID Verification:** You'll need a government-issued ID (e.g., Driver's License, Passport) for either a company officer or an authorized representative. A signed LOA from the officer if the ID is not of the officer. You can find the [LOA template here](#).

Refer [here](#) for detailed information on the preparation of your application.

Server-Side Integration

In order to start an embedded session for a new sender-id registration, a request to the create registration endpoint is required.

There's an additional endpoint that can be used to start new sessions for draft or rejected bundles.

Create Registration

None

POST <https://numbers.twilio.com/v1/SenderIdRegistrations>

- Authentication: Twilio REST API Basic Auth (Account SID : Auth Token)

- The Account SID from the credentials is the account the registration will be linked to.

Request Body

Field	Required	Type	Max Length	Description
regulationId	Required	string	34	The regulation for this registration. Always "RNa8ade60e2a607e62a802f4e6facc887a"
regulationVersion	Required	integer	—	The regulation version. Always "1"
friendlyName	Required	string	255	Human-readable name for the registration.
statusNotificationEmail	Optional	string (email)	255	Email address for registration status notifications.
statusCallbackUrl	Optional	string (uri)	255	URL where Twilio will send HTTP POST notifications when registration status changes.
comments	Optional	string	255	Additional comments about the registration.
themeSetId	Optional	string	255	Theme ID for the Compliance Embeddable UI. Relevant for ISVs who have

Field	Required	Type	Max Length	Description
				requested customized font/color. To request a customized theme, contact Twilio.
<code>data</code>	Required	object	—	Registration data.

Data object

The `data` object contains registration data. Required fields must be provided at creation.

Optional fields pre-populate the corresponding fields in the embeddable form. All fields marked "Collected in Embeddable = Yes" are shown to and validated by the end customer in the embeddable, regardless of whether they were prefilled.

Field	Required	Type	Collected in Embeddable	Description	Constraints
Sender ID					
<code>alphanumericSender.senderId</code>	Required	String	No	The alphanumeric sender ID value. Validated at creation against the Sender ID format required by the regulation.	2-11 characters, ASCII 32-126 only, among other checks.
<code>alphanumericSender.proofOfSenderId</code>	Optional	Enum (String)	Yes	How the sender ID is proven to be associated with the business.	"Company Extract", "Australian Business Names Register", ".au Domain",

Field	Required	Type	Collected in Embeddable	Description	Constraints
					"Australian Trademark", "International Trademark", or "Other"
Business					
<code>business.businessIdentity</code>	Required	Enum (String)	No	Business customer type.	"DIRECT" or "ISV"
<code>business.isSubassigned</code>	Required	Enum (String)	No	Whether the sender ID will be subassigned to ISV's clients.	"YES" or "NO"
<code>business.headquartersCountry</code>	Required	String	No	ISO 3166-1 alpha-2 code for business global headquarters.	2 chars (e.g., "AU")
<code>business.businessName</code>	Optional	String	Yes	Legal name of the business.	Max 255 chars
<code>business.businessType</code>	Optional	Enum (String)	Yes	Legal structure of the business.	e.g., "Private Company", "Sole Trader", "Partnership". See regulation for full list.
<code>business.businessRegistrationNumber</code>	Optional	String	Yes	Business registration number (e.g., ABN, ACN).	Max 21 chars
<code>business.businessWebsite</code>	Optional	String	Yes	Business website URL.	URL format

Field	Required	Type	Collected in Embeddable	Description	Constraints
<code>business.businessRegistrationCountry</code>	Optional	String	Yes	ISO 3166-1 alpha-2 code for the country where the business is legally registered.	2 chars (e.g., "AU", "US")
<code>business.registrationAuthority</code>	Optional	String	Yes	The registration authority (e.g., "ACN", "ABN").	
<code>business.telephoneNumber</code>	Optional	String	Yes	Business telephone number.	E.164 format recommended
<code>business.tradeName</code>	Optional	String	Yes	Trade name or "Doing Business As" name, if different from the legal business name.	Max 255 chars
Use Case					
<code>useCase.category</code>	Required	Enum (String)	No	Purpose of SMS messages.	"PROMOTIONAL" or "TRANSACTIONAL"
<code>useCase.useCaseDescription</code>	Optional	String	Yes	Description of how the sender ID will be used.	
<code>useCase.sampleMessage</code>	Optional	String	Yes	Example message that will be sent	Max 160 chars

Field	Required	Type	Collected in Embeddable	Description	Constraints
				using the sender ID.	
<code>useCase.averageMessageVolumePerMonth</code>	Optional	Enum (String)	Yes	Estimated monthly SMS volume.	"1-1000", "1000-10000", "10000-100000", or "100000+"
Authorized Representative					
<code>authorizedRepresentative.firstName</code>	Optional	String	Yes	Authorized representative first name.	
<code>authorizedRepresentative.lastName</code>	Optional	String	Yes	Authorized representative last name.	
<code>authorizedRepresentative.email</code>	Optional	String	Yes	Authorized representative email address.	Email format
<code>authorizedRepresentative.phoneNumber</code>	Optional	String	Yes	Authorized representative phone number.	E.164 format recommended
Officer					
<code>officer.firstName</code>	Optional	String	Yes	Company officer first name. The officer is the person listed on the business registration extract who authorizes the registration.	

Field	Required	Type	Collected in Embeddable	Description	Constraints
<code>officer.lastName</code>	Optional	String	Yes	Company officer last name.	
<code>officer.email</code>	Optional	String	Yes	Company officer email address.	Email format
Business Address					
<code>businessAddress.street</code>	Optional	String	Yes	Street address of the business.	
<code>businessAddress.streetSecondary</code>	Optional	String	Yes	Secondary street address (e.g., apartment, suite, unit).	
<code>businessAddress.city</code>	Optional	String	Yes	City.	
<code>businessAddress.region</code>	Optional	String	Yes	State, province, or territory (e.g., "NSW", "VIC").	
<code>businessAddress.postalCode</code>	Optional	String	Yes	Postal code.	
<code>businessAddress.isoCountry</code>	Optional	String	Yes	ISO 3166-1 alpha-2 country code for the business address.	2 chars (e.g., "AU")

Note: All fields with "Collected in Embeddable = Yes" are displayed in the embeddable form for the end customer to review, edit, and submit. When prefilled via the API, the values pre-populate the form fields but are still shown to and validated by the end customer.

Response Body

Field	Type	Description
id	string	Registration identifier. Use this for the EmbeddedSessions endpoint (token refresh, resubmission).
regulationId	string	The regulation ID for this registration.
regulationVersion	integer	The regulation version.
friendlyName	string	The friendly name provided in the request.
status	string	Registration status. Always "DRAFT" on creation.
statusNotificationEmail	string	Email echoed from request.
statusCallbackUrl	string	Callback URL echoed from request.
dateCreated	string	Timestamp of creation.
dateUpdated	string	Timestamp of last update.
data	object	Registration data echoed from the request.
embeddedSession	object	Embedded session data.

embeddedSession object

Field	Type	Description
embeddedSession.sessionId	string	Session ID for the compliance embeddable. Pass to the SDK as <code>inquiryId</code> .
embeddedSession.sessionToken	string	Ephemeral session token. Pass to the SDK as <code>inquirySessionToken</code> .

Sample Request

Shell

```
curl -X POST 'https://numbers.twilio.com/v1/SenderIdRegistrations' \
--header 'Content-Type: application/json' \
--data-raw '{
  "regulationId": "RNXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "regulationVersion": 1,
  "friendlyName": "AU Registration - MyBrand",
  "statusNotificationEmail": "registrations@example.com",
  "statusCallbackUrl": "https://example.com/webhooks/sender-id-status",
  "themeSetId": "your-theme-id",
  "data": {
    "alphanumericSender": {
      "senderId": "MyBrand",
      "proofOfSenderId": "Company Extract"
    },
    "business": {
      "businessIdentity": "ISV",
      "isSubassigned": "NO",
      "headquartersCountry": "AU",
      "businessName": "Acme Corp Pty Ltd",
      "businessType": "Private Company",
      "businessRegistrationNumber": "123456789",
      "businessWebsite": "https://www.acmecorp.com.au",
      "businessRegistrationCountry": "AU",
      "registrationAuthority": "ACN",
      "telephoneNumber": "+61212345678",
      "tradeName": "Acme Messaging"
    },
    "useCase": {
      "category": "TRANSACTIONAL",
      "useCaseDescription": "Order confirmation and delivery
notifications for e-commerce customers",
      "sampleMessage": "Hi {name}, your order #{orderId} has been shipped
and will arrive by {date}.",
      "averageMessageVolumePerMonth": "10000-100000"
    },
    "authorizedRepresentative": {
      "firstName": "Jane",
      "lastName": "Smith",
      "email": "jane.smith@acmecorp.com.au",
      "phoneNumber": "+61412345678"
    }
  },
}
```

```

      "officer": {
        "firstName": "John",
        "lastName": "Doe",
        "email": "john.doe@acmecorp.com.au"
      },
      "businessAddress": {
        "street": "123 Collins Street",
        "streetSecondary": "Level 5",
        "city": "Melbourne",
        "region": "VIC",
        "postalCode": "3000",
        "isoCountry": "AU"
      }
    }
  }
}' \
--user $TWILIO_ACCOUNT_SID:$TWILIO_AUTH_TOKEN

```

Sample Response (201 Created)

```

JSON
{
  "id": "BUXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "regulationId": "RNXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "regulationVersion": 1,
  "friendlyName": "AU Registration - MyBrand",
  "status": "DRAFT",
  "statusNotificationEmail": "registrations@example.com",
  "statusCallbackUrl": "https://example.com/webhooks/sender-id-status",
  "dateCreated": "2026-02-17T10:00:00Z",
  "dateUpdated": "2026-02-17T10:00:00Z",
  "data": {
    "alphanumericSender": {
      "senderId": "MyBrand",
      "proofOfSenderId": "Company Extract"
    },
    "business": {
      "businessIdentity": "ISV",
      "isSubassigned": "NO",
      "headquartersCountry": "AU",
      "businessName": "Acme Corp Pty Ltd",
      "businessType": "Private Company",

```

```
        "businessRegistrationNumber": "123456789",
        "businessWebsite": "https://www.acmecorp.com.au",
        "businessRegistrationCountry": "AU",
        "registrationAuthority": "ACN",
        "telephoneNumber": "+61212345678",
        "tradeName": "Acme Messaging"
    },
    "useCase": {
        "category": "TRANSACTIONAL",
        "useCaseDescription": "Order confirmation and delivery
notifications for e-commerce customers",
        "sampleMessage": "Hi {name}, your order #{orderId} has been shipped
and will arrive by {date}.",
        "averageMessageVolumePerMonth": "10000-100000"
    },
    "authorizedRepresentative": {
        "firstName": "Jane",
        "lastName": "Smith",
        "email": "jane.smith@acmecorp.com.au",
        "phoneNumber": "+61412345678"
    },
    "officer": {
        "firstName": "John",
        "lastName": "Doe",
        "email": "john.doe@acmecorp.com.au"
    },
    "businessAddress": {
        "street": "123 Collins Street",
        "streetSecondary": "Level 5",
        "city": "Melbourne",
        "region": "VIC",
        "postalCode": "3000",
        "isoCountry": "AU"
    }
},
"embeddedSession": {
    "sessionId": "inq_XXXXXXXXXXXXXXXXXXXXXXX",
    "sessionToken": "eyJhbGciOiJIUzI1NiIs..."
}
}
```

Create a new Embedded Session

None

POST <https://numbers.twilio.com/v1/SenderIdRegistrations/{id}/EmbeddedSessions>

```
{
  "themeSetId": "id"
}
```

- Authentication: Twilio REST API Basic Auth (Account SID : Auth Token)
- The Account SID from the credentials must be the same account that owns the registration.
- `{id}` refers to the `id` value returned in the Create Registration endpoint.

Use Cases

1. **Registration in DRAFT status (previously started)** — Creates a new session on the existing inquiry. Previously submitted data is retained.
2. **Registration in TWILIO_REJECTED status** — Creates a new inquiry for resubmission. Rejection reasons and previously submitted data are shown in the embeddable.
3. **Registration in other statuses (IN_REVIEW, APPROVED)** — Returns a status code 409 (Conflict).

Request Body

Field	Required	Type	Max Length	Description
<code>themeSetId</code>	Optional	string	255	Theme ID for the Compliance Embeddable UI. Overrides the theme set during registration creation for this session.

Sample Request

```
Shell
curl -X POST
'https://numbers.twilio.com/v1/SenderIdRegistrations/BUXXXXXXXXXXXXXXXXXXXXXX
XXXXXXXX/EmbeddedSessions' \
--header 'Content-Type: application/json' \
--data-raw '{
  "themeSetId": "your-theme-id"
}' \
--user $TWILIO_ACCOUNT_SID:$TWILIO_AUTH_TOKEN
```

Response (201 Created)

```
JSON
{
  "id": "BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "sessionId": "inq_XXXXXXXXXXXXXXXXXXXXX",
  "sessionToken": "eyJhbGciOiJIUzI1NiIs..."
}
```

Error Codes

The following error codes may be returned by the API endpoints. Use the error code to programmatically determine the type of error and the appropriate recovery action.

Error Response Format

All error responses follow the standard Twilio error response format:

```
JSON
{
  "message": "Invalid Sender ID for target country: ...",
  "more_info": "https://www.twilio.com/docs/errors/21920",
  "status": 400
}
```

POST /v1/SenderIdRegistrations

HTTP Status	Error Message	Recovery Action
400	Invalid parameter: {details}	Fix the request body. Check the error message for the specific field that failed validation.
400	Invalid Sender ID for target country: {details}	The sender ID does not meet the target country requirements. For AU: 2-11 alphanumeric characters, must contain at least one letter. Prompt the end-customer to change their sender ID.
400	Regulation '{regulationId}' is not supported for sender ID registration	The regulation is not currently supported. Check documentation for supported regulations.

POST /v1/SenderIdRegistrations/{id}/EmbeddedSessions

HTTP Status	Error Message	Recovery Action
400	Invalid parameter	Check the request body for invalid fields.
404	Not Found	The registration ID does not exist. Verify the <code>id</code> value.
409	Registration not in editable state. Current status: {status}	Only registrations in DRAFT or TWILIO_REJECTED status can create new sessions. If IN_REVIEW, wait for the review to complete. If TWILIO_APPROVED, the registration is finalized.

Common Error Codes

These standard Twilio error codes may be returned by any endpoint.

HTTP Status	Error Message	Description
400	Invalid parameter: {details}	A request parameter is missing or invalid. Check the error message for the specific field.

HTTP Status	Error Message	Description
401	Permission Denied	The API credentials (Account SID and Auth Token) are missing or invalid.
403	Permission Denied	The account does not have permission to access this resource.
404	Not Found	The requested resource does not exist. Verify the resource identifier.
500	Internal Server Error	An unexpected error occurred. Retry the request with exponential backoff. If the error persists, contact Twilio support.

Handling Status Callbacks

If a `statusCallbackUrl` URL was provided when the registration was created, Twilio will send an HTTP **POST** request (form-encoded) to that URL whenever the registration status changes.

Callback Payload

The callback is sent as `application/x-www-form-urlencoded` with the following fields:

Name	Description
AccountSid	The Twilio Account SID associated with the registration.
BundleSid	The BU-prefixed SID identifying the registration.
Status	The new registration status. See status values below.
FailureReason	The reason for rejection, if applicable. If there is more than one reason, this will be a semicolon separated list. Empty string if not applicable.

Registration Status Values

Status	Description
DRAFT	Registration has been created but the embeddable has not been completed.
IN_REVIEW	Registration is being reviewed by Twilio.
TWILIO_APPROVED	Registration has been approved.
TWILIO_REJECTED	Registration has been rejected. See FailureReason for details.

Status Transitions

- [DRAFT](#) → [IN_REVIEW](#) (after the customer completes the embeddable)
- [IN_REVIEW](#) → [TWILIO_APPROVED](#) (registration approved)
- [IN_REVIEW](#) → [TWILIO_REJECTED](#) (registration rejected)
- [TWILIO_REJECTED](#) → [IN_REVIEW](#) (registration resubmitted)

Recommended Actions

- On **TWILIO_APPROVED**: The registration is complete.
 - On **TWILIO_REJECTED**: Use the Create a New Embeddable Session endpoint to obtain a new session token and re-render the embeddable so the customer can correct and resubmit their registration.
-

Client-Side Integration

After initializing a Sender ID Registration using the above server-side integration, you may render the embedded experience in your website using the TwilioComplianceEmbed React client.

Installation

- Install the package in your React project using your preferred package manager
 - **NPM:** `npm install @twilio/twilio-compliance-embed`
 - **Yarn:** `yarn add @twilio/twilio-compliance-embed`

Usage

React

The embeddable UI can be rendered in your site with the following code snippet:

JavaScript

```
import * as React from "react";
import { TwilioComplianceEmbed } from "@twilio/twilio-compliance-embed";

const SenderIdRegistrationEmbed = () => {
  return (
    <TwilioComplianceEmbed
      inquiryId='<your inquiry_id from server-side request>'
      inquirySessionToken='<your inquiry_session_token from server-side
request>'
    />
  );
};
```

Note: launching the embeddable UI may take a few seconds to load & render.

Available Properties

Name	Type	Description
inquiryId	(Required) String	A valid inquiryId returned by the server-side Create Registration or Create a new Embedded Session call.
inquirySessionToken	(Required) String	A valid inquirySessionToken returned by the server-side Create Registration or Create a new Embedded Session call.
onInquirySubmitted	() => void	An event handler that fires when the user has completed the embeddable process. This event is fired when the user lands on the last screen.

Name	Type	Description
onCancel	() => void	An event handler that fires when the user has canceled the embeddable process.
onError	() => void	An event handler that fires when an unexpected error occurs during the embeddable process.
onReady	() => void	An event handler that fires when the embeddable UI has finished loading.
onEvent	() => void	An event handler that fires at certain points. It takes two arguments, an eventName string and a metadata object. eventName Description page-change page-change events receive a metadata object containing the following fields: - metadata.metadata.pageHeight : height in pixels of the content on the current page
widgetPadding	{ top?: number; bottom?: number; left?: number; right?: number; }	Allows customizing the internal padding inside the iframe. If this is not passed it will use the default values <code>{top: 74, left: 24, right: 24, bottom: 24}</code> Value should be an object with shape <code>{top?: number; bottom?: number; left?: number; right?: number;}</code>

Sample Usage

JavaScript

```
import * as React from "react";
import { Spinner } from "@twilio-paste/core/spinner";
import { TwilioComplianceEmbed } from "@twilio/twilio-compliance-embed";

function MySenderIdRegistration() {
  const [data, setData] = React.useState(null);
  const [isLoading, setLoading] = React.useState(true);

  React.useEffect(() => {
    fetch("/my/backend/server/api/sender-id-registrations", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        regulationId: "RNXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
        regulationVersion: 1,
        friendlyName: "AU Registration - MyBrand",
        statusNotificationEmail: "registrations@example.com",
        statusCallbackUrl: "https://example.com/webhooks/sender-id-status",
        data: {
          alphanumericSender: {
            senderId: "MyBrand"
          },
          business: {
            businessIdentity: "ISV",
            isSubassigned: "NO",
            headquartersCountry: "AU",
            businessName: "Acme Corp Pty Ltd",
            businessRegistrationNumber: "123456789"
          },
          useCase: {
            category: "TRANSACTIONAL"
          },
          authorizedRepresentative: {
            firstName: "Jane",
            lastName: "Smith",
            email: "jane.smith@acmecorp.com.au",
            phoneNumber: "+61412345678"
          }
        }
      })
    })
      .then((res) => res.json())
      .then((data) => {
        setData(data);
      })
  });
}
```

```

        setLoading(false);
    });
}, []);

return !isLoading ? (
    <TwilioComplianceEmbed
        inquiryId={data.embeddedSession.sessionId}
        inquirySessionToken={data.embeddedSession.sessionToken}
        onReady={() => {
            console.log("Ready!");
        }}
        onInquirySubmitted={() => {
            console.log("Registration submitted");
        }}
        widgetPadding={{top: 0, left: 100, right: 100, bottom: 0}}
        onEvent={(eventName, metadata) => {
            if (eventName === 'page-change') {
                console.log("Height in pixels", metadata?.metadata?.pageHeight);
            }
        }}
    />
) : (
    <Spinner decorative={false} title="Loading" />
);
}

export default MySenderIdRegistration;

```

Session Expiration

Embedded Sessions expire after a maximum of 24 hours. Once a session has expired (without additional handling) client-side users will be kicked out of the registration process until a new session is provisioned. Handling of this scenario can be orchestrated client-side to improve user experience.

Session expirations can be detected by listening for an event from the `onError` handler with the following event body:

JavaScript

```
{ status: 409, code: 'unauthenticated' }
```

In the event of session expiration, the client will need to create a new Embedded Session (following the [Create a new Embedded Session](#) steps above) and optionally display a loading state to avoid displaying any errors or “flashing” to the user.

A sample implementation may look like:

JavaScript

...

```
const createNewEmbeddedSession = async () => {
  const id = data.id; // Registration ID from the initial response
  const response = await fetch(
    `/my/backend/server/api/sender-id-registrations/${id}/embedded-session`
  );

  return response.json();
};

const handleSessionExpiration = async () => {
  setLoading(true);
  const newSession = await createNewEmbeddedSession();
  setData({ ...data, embeddedSession: newSession });
  setLoading(false);
};

const errorHandler = (error) => {
  const { status, code } = error;
  if (status === 409 && code === "unauthenticated") {
    handleSessionExpiration();
  }

  // Additional error handling...
};

return !isLoading ? (
  <TwilioComplianceEmbed
    ...
    onError={errorHandler}
    ...
```

```
    />
  ) : (
    <Spinner decorative={false} title="Loading" />
  );

  ...

```

Read Registration Data

After a registration has been submitted through the embeddable, you can read back the registration data — for example, to display submitted details in your portal, to power internal auditing workflows, or to verify what was collected before acting on a status callback.

The registration `id` (BU-prefixed) returned from Create Registration corresponds to a TrustProduct in Twilio's [Trust Hub API](#). Reading the registration data requires multiple API calls because the data is stored across several entity types (EndUsers and SupportingDocuments) assigned to the TrustProduct.

Overview of API Calls

Step	API Call	Purpose
1	GET <code>https://trusthub.twilio.com/v1/TrustProducts/{TrustProductSid}</code>	Fetch the registration bundle and its status
2	GET <code>https://trusthub.twilio.com/v1/TrustProducts/{TrustProductSid}/EntityAssignments</code>	List all assigned EndUsers and Documents
3	GET <code>https://trusthub.twilio.com/v1/EndUsers/{Sid}</code>	Fetch identity data (for IT-prefixed sids)
4	GET <code>https://trusthub.twilio.com/v1/SupportingDocuments/{Sid}</code>	Fetch document data (for RD-prefixed sids)

Step	API Call	Purpose
5	GET <code>https://api.twilio.com/2010-04-01/Accounts/{AccountSid}/Addresses/{AddressSid}.json</code>	Fetch address details (from business_address document)

- Steps 1–4 use `trusthub.twilio.com` as the base host.
- Step 5 uses `api.twilio.com`.
- All endpoints use Twilio REST API Basic Auth (Account SID : Auth Token), the same authentication as the Create Registration endpoint.

Step 1: Fetch the TrustProduct

Retrieve the top-level registration bundle. The `{TrustProductSid}` is the `id` value (BU-prefixed) returned from the Create Registration endpoint.

```
Shell
curl -X GET
'https://trusthub.twilio.com/v1/TrustProducts/BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX' \
--user $TWILIO_ACCOUNT_SID:$TWILIO_AUTH_TOKEN
```

```
JSON
{
  "sid": "BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "account_sid": "ACXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "policy_sid": "RNXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "friendly_name": "AU Registration - MyBrand",
  "status": "in-review",
  "email": "registrations@example.com",
  "status_callback": "https://example.com/webhooks/sender-id-status",
  "valid_until": null,
  "date_created": "2026-02-17T10:00:00Z",
  "date_updated": "2026-02-17T10:05:00Z",
  "url":
'https://trusthub.twilio.com/v1/TrustProducts/BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX',
  "links": {
```

```

        "trust_products_entity_assignments":
        "https://trusthub.twilio.com/v1/TrustProducts/BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX/EntityAssignments",
        "trust_products_evaluations":
        "https://trusthub.twilio.com/v1/TrustProducts/BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX/Evaluations",
        "trust_products_channel_endpoint_assignment":
        "https://trusthub.twilio.com/v1/TrustProducts/BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX/ChannelEndpointAssignments"
    }
}

```

Reference: [Fetch a TrustProduct Resource](#)

Step 2: List Entity Assignments

List all EndUsers and SupportingDocuments assigned to the registration. Each assignment contains an `object_sid` that identifies the assigned entity.

Shell

```

curl -X GET
'https://trusthub.twilio.com/v1/TrustProducts/BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX/EntityAssignments' \
--user $TWILIO_ACCOUNT_SID:$TWILIO_AUTH_TOKEN

```

JSON

```

{
  "results": [
    {
      "sid": "BVXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
      "trust_product_sid": "BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
      "account_sid": "ACXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
      "object_sid": "ITXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
      "date_created": "2026-02-17T10:01:00Z",
      "url":
        "https://trusthub.twilio.com/v1/TrustProducts/BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX/EntityAssignments/BVXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
    },
    {

```

```

        "sid": "BVYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYY",
        "trust_product_sid": "BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
        "account_sid": "ACXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
        "object_sid": "ITYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYY",
        "date_created": "2026-02-17T10:01:00Z",
        "url":
"https://trusthub.twilio.com/v1/TrustProducts/BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX/EntityAssignments/BVYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYY"
    },
    {
        "sid": "BVZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZ",
        "trust_product_sid": "BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
        "account_sid": "ACXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
        "object_sid": "RDXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
        "date_created": "2026-02-17T10:02:00Z",
        "url":
"https://trusthub.twilio.com/v1/TrustProducts/BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX/EntityAssignments/BVZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZ"
    }
],
"meta": {
    "page": 0,
    "page_size": 50,
    "first_page_url":
"https://trusthub.twilio.com/v1/TrustProducts/BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX/EntityAssignments?PageSize=50&Page=0",
    "previous_page_url": null,
    "url":
"https://trusthub.twilio.com/v1/TrustProducts/BUXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX/EntityAssignments?PageSize=50&Page=0",
    "next_page_url": null,
    "key": "results"
}
}

```

- **object_sid** values with an **IT** prefix are EndUser entities (identity data).
- **object_sid** values with an **RD** prefix are SupportingDocument entities (documents and address references).

Reference: [List Entity Assignments](#)

Step 3: Fetch EndUser Details

For each **IT**-prefixed **object_sid**, fetch the EndUser to retrieve identity data. The **type** field indicates which identity this EndUser represents, and the **attributes** object contains the stored fields.

Shell

```
curl -X GET
'https://trusthub.twilio.com/v1/EndUsers/ITXXXXXXXXXXXXXXXXXXXXXXXXXXXX' \
--user $TWILIO_ACCOUNT_SID:$TWILIO_AUTH_TOKEN
```

Example response (type: **business):**

JSON

```
{
  "sid": "ITXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "account_sid": "ACXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "friendly_name": "AU Registration - MyBrand",
  "type": "business",
  "date_created": "2026-02-17T10:01:00Z",
  "date_updated": "2026-02-17T10:05:00Z",
  "attributes": {
    "business_name": "Acme Corp Pty Ltd",
    "business_type": "Private Company",
    "business_registration_number": "123456789",
    "business_website": "https://www.acmecorp.com.au",
    "phone_number": "+61212345678",
    "headquarters_country": "AU",
    "country": "AU",
    "business_identity": "isv_reseller_or_partner",
    "is_subassigned": "NO",
    "trade_name": "Acme Messaging"
  },
  "url":
  "https://trusthub.twilio.com/v1/EndUsers/ITXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
}
```

Reference: [Fetch an EndUser Resource](#)

Step 4: Fetch SupportingDocument Details

For each **RD**-prefixed **object_sid**, fetch the SupportingDocument. The **type** field indicates the document category, and the **attributes** object contains stored fields or references.

```
Shell
curl -X GET
'https://trusthub.twilio.com/v1/SupportingDocuments/RDXXXXXXXXXXXXXXXXXXXXXXXXX
XXXXXX' \
--user $TWILIO_ACCOUNT_SID:$TWILIO_AUTH_TOKEN
```

Example response (type: **business_address**):

```
JSON
{
  "sid": "RDXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "account_sid": "ACXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "friendly_name": "AU Registration - MyBrand",
  "type": "business_address",
  "status": "APPROVED",
  "date_created": "2026-02-17T10:02:00Z",
  "date_updated": "2026-02-17T10:05:00Z",
  "attributes": {
    "address_sids": "ADXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
  },
  "mime_type": null,
  "url":
'https://trusthub.twilio.com/v1/SupportingDocuments/RDXXXXXXXXXXXXXXXXXXXXXXXXX
XXXXXX'
}
```

Reference: [Fetch a SupportingDocument Resource](#)

Step 5: Fetch Address Details

If a **business_address** SupportingDocument is found, use the **address_sids** value to fetch the full address from the Twilio Addresses API.

Shell

```
curl -X GET
'https://api.twilio.com/2010-04-01/Accounts/ACXXXXXXXXXXXXXXXXXXXXXXXXXXXX/
Addresses/ADXXXXXXXXXXXXXXXXXXXXXXXXXXXX.json' \
--user $TWILIO_ACCOUNT_SID:$TWILIO_AUTH_TOKEN
```

JSON

```
{
  "sid": "ADXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "account_sid": "ACXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "friendly_name": "Acme Corp Pty Ltd",
  "customer_name": "Acme Corp Pty Ltd",
  "street": "123 Collins Street",
  "street_secondary": "Level 5",
  "city": "Melbourne",
  "region": "VIC",
  "postal_code": "3000",
  "iso_country": "AU",
  "emergency_enabled": false,
  "validated": false,
  "verified": false,
  "date_created": "Tue, 17 Feb 2026 10:02:00 +0000",
  "date_updated": "Tue, 17 Feb 2026 10:02:00 +0000",
  "uri":
"/2010-04-01/Accounts/ACXXXXXXXXXXXXXXXXXXXXXXXXXXXX/Addresses/ADXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXX.json"
}
```

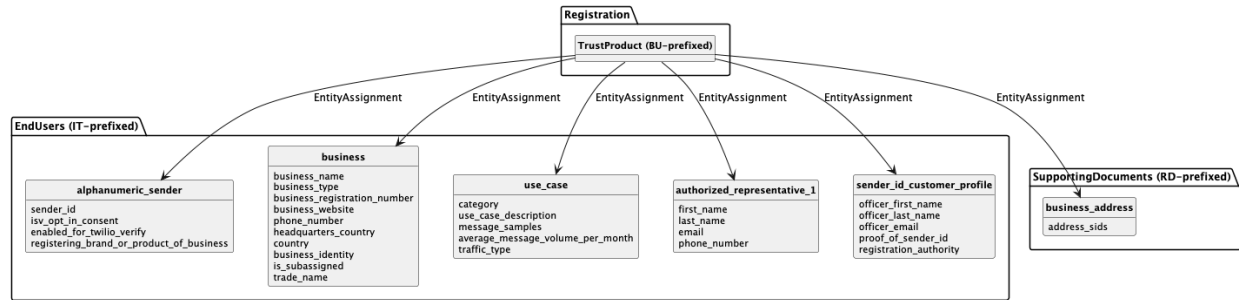
Note: The Address API uses RFC 2822 date format (e.g., "Tue, 17 Feb 2026 10:02:00 +0000"), unlike the Trust Hub APIs which use ISO 8601 format.

Reference: [Fetch an Address Resource](#)

Entity Type Reference

The following diagram and tables show which EndUser and SupportingDocument types you can expect in an AU Sender ID registration and the fields stored in each.

Entity Overview



EndUser Field Reference

Each EndUser entity is fetched via `GET /v1/EndUsers/{IT-sid}`. The `type` field identifies the entity, and `attributes` contains the stored fields.

alphanumeric_sender

Sender ID registration details.

Field	Type	Description
<code>sender_id</code>	String	The alphanumeric sender ID value (e.g., "MyBrand"). 2-11 characters.
<code>isv_opt_in_consent</code>	Boolean	Whether ISV opt-in consent was given.
<code>enabled_for_twilio_verify</code>	Boolean	Whether Twilio Verify is enabled for this sender ID.
<code>registering_brand_or_product_of_business</code>	Boolean	Whether the sender ID represents a brand or product of the business.

business

Business information for the registering entity.

Field	Type	Description
<code>business_name</code>	String	Legal name of the business.

Field	Type	Description
business_type	String	Legal structure (e.g., "Private Company", "Sole Trader", "Partnership").
business_registration_number	String	Business registration number (e.g., ABN, ACN).
business_website	String	Business website URL.
phone_number	String	Business telephone number.
headquarters_country	String	ISO 3166-1 alpha-2 code for business global headquarters (e.g., "AU").
country	String	ISO 3166-1 alpha-2 code for the target/recipient country (e.g., "AU").
business_identity	String	Business customer type (e.g., "isv_reseller_or_partner", "direct_customer").
is_subassigned	String	Whether the sender ID will be subassigned to ISV's clients ("YES" or "NO").
trade_name	String	Trade name or "Doing Business As" name, if different from the legal business name.

use_case

Use case details for the sender ID.

Field	Type	Description
category	String	Purpose of SMS messages ("PROMOTIONAL" or "TRANSACTIONAL").

Field	Type	Description
<code>use_case_description</code>	String	Description of how the sender ID will be used.
<code>message_samples</code>	String	Example message that will be sent using the sender ID.
<code>average_message_volume_per_month</code>	String	Estimated monthly SMS volume ("1-1000", "1000-10000", "10000-100000", or "100000+").
<code>traffic_type</code>	String	Traffic scope ("domestic" or "international").

`authorized_representative_1`

The person authorized to submit the registration.

Field	Type	Description
<code>first_name</code>	String	Authorized representative first name.
<code>last_name</code>	String	Authorized representative last name.
<code>email</code>	String	Authorized representative email address.
<code>phone_number</code>	String	Authorized representative phone number.

`sender_id_customer_profile`

AU-specific fields for officer details and proof of sender ID.

Field	Type	Description
<code>officer_first_name</code>	String	Company officer first name. The officer is the person listed on the business

Field	Type	Description
		registration extract who authorizes the registration.
<code>officer_last_name</code>	String	Company officer last name.
<code>officer_email</code>	String	Company officer email address.
<code>proof_of_sender_id</code>	String	How the sender ID is proven to be associated with the business ("Company Extract", "Australian Business Names Register", ".au Domain", "Australian Trademark", "International Trademark", or "Other").
<code>registration_authority</code>	String	The registration authority (e.g., "ACN", "ABN").

SupportingDocument Field Reference

Each SupportingDocument entity is fetched via `GET /v1/SupportingDocuments/{RD-sid}`. The `type` field identifies the document, and `attributes` contain stored fields or references.

`business_address`

Reference to the business address. Use the `address_sids` value with the Address API (Step 5) to fetch the full address.

Field	Type	Description
<code>address_sids</code>	String	The AD-prefixed SID of the address. Fetch via <code>GET /2010-04-01/Accounts/{AccountSid}/Addresses/{AddressSid}.json</code> .

Security

Exchange of all sensitive information such as credentials, permanent resource identifiers, and PII is designed to occur securely between your website, your server and the Twilio API.

- Your customer initiates the Sender ID Registration embeddable UI in your site.
- Your server will request the `sessionId` & the ephemeral `sessionToken` from the Twilio API using Twilio's REST API authentication mechanism (<https://www.twilio.com/docs/iam/credentials/api>).
- Your server will return the `sessionId` & `sessionToken` to your website to be passed into the TwilioComplianceEmbed client component.
 - **Note:** we recommend enabling CSP (Content Security Policy) as an added layer of security to protect these secrets in the browser.
- The TwilioComplianceEmbed client component will launch and securely load the embeddable experience automatically.